

Manual UX de Mateu

Patrones de diseño para aplicaciones de negocio

VERSIÓN EDITORIAL · V8

Este handbook describe los principios y patrones de UX que Mateu soporta —o debería soportar— para construir aplicaciones de negocio productivas, resilientes y centradas en el workflow. Cada patrón se vincula con la anotación real del framework que lo expresa; donde no existe primitiva todavía, se propone una.

Convertir patrones de UX en primitivas declarativas reutilizables.

● Existe ● Parcial ● Falta primitiva

1 Introducción

Las aplicaciones de backoffice no son productos de marketing ni experiencias de consumo. Son herramientas de trabajo. Su calidad no se mide por la espectacularidad visual sino por la productividad, la seguridad y la reducción de esfuerzo cognitivo.

Los usuarios pasan horas dentro de estos sistemas. Cada clic innecesario, cada filtro perdido y cada cambio de contexto tiene un coste operativo real.

Mateu se diferencia no por seguir modas visuales sino por convertir patrones de UX en **primitivas declarativas reutilizables**. La tesis del framework es que la buena UX de negocio es lo bastante regular como para codificarse una vez, en el backend, y aplicarse de forma consistente sin escribir frontend.

2 Principios UX

Cada principio existe para evitar un coste concreto. No son aspiraciones; son criterios de decisión cuando un patrón admite varias formas.

- 01 Preservar contexto.** El usuario nunca debería perder orientación ni estado mental. Salir de una pantalla para volver a entrar es la forma más cara de perder contexto.
- 02 Minimizar navegación.** Menos pantallas y menos saltos equivalen a más productividad. Componer vence a navegar.
- 03 Keyboard-first.** Los usuarios avanzados viven en el teclado. Si una tarea frecuente exige el ratón, está mal diseñada.
- 04 Workflow over screens.** La unidad de diseño es la tarea, no la pantalla. Se diseña para completar un trabajo, no para mostrar una entidad.
- 05 Complejidad progresiva.** Mostrar complejidad solo cuando aporta valor. Lo avanzado se revela, no se impone.
- 06 Recuperabilidad.** Todo error debe poder corregirse. Confirmar lo destructivo, deshacer lo reversible, no perder nunca el trabajo en curso.
- 07 Consistencia.** Los patrones se repiten de forma predecible. Lo que se aprende en una pantalla sirve en todas.

3 Cómo encaja todo

Patrones, anti-patrones y primitivas que faltan no son listas independientes. Forman un sistema: cada **anti-patrón** es un fallo que uno o más **patrones** evitan; cada patrón sirve a uno o más **principios**; y cada patrón se expresa con una **anotación concreta** de Mateu, o señala una primitiva que falta.

ANTI-PATRÓN	SE EVITA CON	PRINCIPIO
CRUD Tunnel	Split View	Preservar contexto · Minimizar navegación
Modal Hell	Split View · Task-centric page	Preservar contexto
Spinner Prison	Jobs largos (asíncrono)	Recuperabilidad · Preservar contexto
Filter Amnesia	Saved Views (<i>propuesta</i>) · Deep linking	Preservar contexto · Workflow over screens
Desktop Denial	Atajos de teclado	Keyboard-first

En el catálogo que sigue, cada ficha indica su intención, el problema concreto que resuelve, la dirección Mateu (anotación real o propuesta) y la estructura de la vista.

4 Catálogo de patrones

Split View

EXISTE

INTENCIÓN	Trabajar sobre una colección sin abandonar nunca la colección.
PROBLEMA	El flujo listado → detalle → volver obliga a perder la posición en la lista, los filtros y el scroll en cada ida y vuelta. Quien revisa cuarenta pedidos paga ese peaje cuarenta veces. Es la causa raíz del CRUD Tunnel.
DIRECCIÓN	<code>@SplitLayout</code> a nivel de vista, con <code>@MasterDetail</code> sobre el campo de detalle (atributo <code>minHeightWhenDetailVisible</code>). Seleccionar una fila renderiza el detalle en el panel adjacente en lugar de enrutar. Stateless: la selección envía el id y el backend devuelve la metadata del panel.
ESTRUCTURA	Lista con filtros persistentes · detalle · acciones en el detalle.

Filtros · Listado · Detalle

EXISTE

INTENCIÓN	Localizar y operar sobre registros con el mínimo esfuerzo.
PROBLEMA	Un listado sin filtros vivos, sin acciones por fila y sin acciones masivas obliga a navegar a cada registro para hacer cualquier cosa.
DIRECCIÓN	<code>@List</code> para la colección, <code>@Filterable</code> sobre los campos de filtro, <code>@RowAction</code> para acciones por fila y <code>@ListToolbarButton</code> (con <code>rowsSelectedRequired</code>) para acciones masivas. <code>@ColumnWidth</code> y <code>@HiddenInList</code> afinan la rejilla.
ESTRUCTURA	Barra de filtros · rejilla · toolbar de acciones masivas · acciones por fila.

Wizard

EXISTE

INTENCIÓN	Guiar flujos secuenciales con dependencias entre pasos.
PROBLEMA	Un alta donde el paso 3 depende de lo elegido en el 1 se resuelve con un formulario gigante o con saltos sin validación intermedia. El usuario avanza sin saber si puede.
DIRECCIÓN	Cada paso es una vista cuya acción de avance valida y devuelve el siguiente paso; <code>@WizardCompletionAction</code> marca la acción que cierra el flujo. La validación intermedia se fuerza con <code>@Action(validationRequired=true)</code> .
ESTRUCTURA	Indicador de progreso · paso actual · avance validado · cierre.

Entity Picker

EXISTE

INTENCIÓN	Seleccionar relaciones sin abandonar el formulario.
PROBLEMA	Elegir un cliente entre miles, o gestionar líneas hijas, suele empujar a otra pantalla y rompe el formulario en curso.
DIRECCIÓN	Para seleccionar una relación, <code>@Lookup</code> con su <code>search</code> (un <code>LookupOptionsSupplier</code>) y su <code>label</code> (un <code>LabelSupplier</code>). Para gestionar colecciones hijas en línea, <code>@Composition</code> (<code>targetClass</code> , <code>repositoryClass</code> , <code>foreignKeyField</code>).
ESTRUCTURA	Campo con búsqueda incremental · resultados · vista previa · selección, dentro del formulario.

Workspace

COMPOSICIÓN

INTENCIÓN	Un entorno multipanel donde conviven varias vistas relacionadas a la vez.
PROBLEMA	Tareas complejas exigen mirar varias cosas simultáneamente (cola + detalle + contexto). Resolverlas saltando entre pantallas multiplica la carga cognitiva.
VS SPLIT VIEW	Split View son dos paneles maestro-detalle de la misma colección. Workspace es la composición de varias vistas heterogéneas en una pantalla.
DIRECCIÓN	Por composición: <code>@SplitLayout</code> , <code>@VerticalLayout</code> / <code>@HorizontalLayout</code> , <code>@Tabs</code> / <code>@Tab</code> , <code>@Accordion</code> / <code>@Section</code> y <code>@KPI</code> en cabecera. No requiere primitiva nueva, sino una guía de composición.
ESTRUCTURA	Cabecera de KPIs · navegación/cola · panel de trabajo · panel de contexto · acciones.

Task-centric page

COMPOSICIÓN

INTENCIÓN	Una pantalla diseñada alrededor de una tarea, no de una entidad.
PROBLEMA	«Editar pedido» obliga al usuario a saber qué campos tocar y en qué orden. «Confirmar envío» presenta exactamente lo necesario para ese trabajo.
DIRECCIÓN	Una vista que expone solo los campos relevantes más las acciones que la completan (<code>@Action</code> , <code>@MainAction</code>); lo irreversible lleva <code>@Action(confirmationRequired=true)</code> . Es el principio <i>Workflow over screens</i> aplicado al diseño de la clase, no una primitiva nueva.
ESTRUCTURA	Contexto mínimo · campos de la tarea · acción principal destacada · secundarias.

Jobs largos

EXISTE

INTENCIÓN	Dar UX digna a procesos asíncronos sin congelar la interfaz.
PROBLEMA	Un proceso de varios segundos o minutos que bloquea la pantalla entera es el Spinner Prison: el usuario no puede hacer nada y no sabe si el sistema sigue vivo.
DIRECCIÓN	<code>@Action(background=true)</code> lanza el trabajo sin bloquear; <code>sse=true</code> emite progreso por server-sent events. Para sondear estado, <code>@Trigger(type=OnLoad, times=N, timeoutMillis=T)</code> ; <code>@Trigger(OnSuccess/OnError)</code> reacciona al desenlace. Un <code>ProgressBar</code> o un <code>@Status</code> reflejan el avance.
ESTRUCTURA	Disparador no bloqueante · progreso vivo · feedback de éxito/error · reintento.

Timeline

PARCIAL

INTENCIÓN	Hacer auditable la historia de un registro o proceso.
PROBLEMA	Sin historial visible, el usuario no sabe qué pasó, cuándo ni quién lo hizo. La auditabilidad se convierte en una consulta a soporte o a la base de datos.
DIRECCIÓN	Hoy existe <code>@Status</code> / <code>@StatusMapping</code> / <code>@State</code> para el estado actual (<code>NONE</code> , <code>INFO</code> , <code>SUCCESS</code> , <code>WARNING</code> , <code>DANGER</code>) y el tipo <code>Bpmn</code> para diagramas de proceso. No existe un feed cronológico de eventos: ver la propuesta <code>@Timeline</code> en §6.
ESTRUCTURA	Eventos ordenados en el tiempo · autor · marca temporal · estado en cada hito.

Autosave + Dirty State

FALTA PRIMITIVA

INTENCIÓN	Eliminar la ansiedad de perder trabajo y la duda de si se guardó.
PROBLEMA	El usuario rellena un formulario largo, se le va la sesión o cierra la pestaña, y pierde todo. O duda de si «Guardar» llegó a aplicar.
DIRECCIÓN	Componible hoy con <code>@Trigger(type=OnValueChange, calledActionId="save")</code> sobre una acción de guardado idempotente. Falta el indicador de estado <i>dirty</i> y una convención de autosave con debounce: ver la propuesta <code>@Autosave</code> en §6.
ESTRUCTURA	Edición continua · estado (guardado / guardando / sin guardar) · recuperación tras recarga.

5 Anti-patrones UX

Cada anti-patrón se nombra para reconocerlo y se cierra con el patrón que lo evita.

Modal Hell

Modales anidados que destruyen la orientación.

Se evita con → Split View y Task-centric page; el modal se reserva para confirmaciones (`@Action(confirmationRequired=true)`), nunca para flujos.

CRUD Tunnel

Listado → detalle → volver, que rompe el flujo.

Se evita con → Split View (`@SplitLayout` + `@MasterDetail`).

Spinner Prison

Bloquear la pantalla completa mientras algo carga.

Se evita con → Jobs largos (`@Action(background=true, sse=true)` + polling por `@Trigger`).

Filter Amnesia

Perder los filtros al navegar.

Se evita con → Saved Views (*propuesta*) y deep linking que codifica el filtro en la URL (`@Route`).

Desktop Denial

Ignorar el teclado y castigar al usuario experto.

Se evita con → atajos de teclado; hoy `@Trigger(type=OnEnter)` cubre el envío con Enter, pero faltan atajos generales (§6).

6 Primitivas que faltan

Las tres piezas que el catálogo necesita y que v3 todavía no tiene como primitiva de primera clase. Se proponen diseños concretos, coherentes con el estilo de anotaciones existente, para discutir.

@Autosave PROPUESTA

Guardado automático con debounce e indicador de estado *dirty*.

```
@Autosave(debounceMillis = 800, action = "save")
public class EditarPedido { ... }
```

- A nivel de clase activa el autosave de los campos editables; a nivel de campo lo restringe.
- El renderer compara el estado del cliente con el último snapshot confirmado para derivar el estado *dirty* y mostrar el indicador (sin guardar / guardando / guardado).
- Requiere que la acción referenciada sea **idempotente** y acepte guardados parciales.
- Composición mínima hoy: `@Trigger(OnValueChange, calledActionId="save")` cubre el disparo, pero no el debounce ni el indicador.

@Timeline PROPUESTA

Feed cronológico de eventos sobre un registro o proceso.

```
@Timeline(source = HistorialPedido.class)
private List<Evento> historial;
```

- Cada evento expone marca temporal, autor, descripción y un `StatusType` para colorear el hito — reutilizando la semántica de `@Status`.
- Complementa a `@State` / `@Status` (estado actual) con la **historia** que falta.
- Para procesos con ramas, el tipo `Bpmn` sigue siendo lo adecuado; `@Timeline` es para historiales lineales y auditoría.

Saved Views PROPUESTA

Persistencia de combinaciones con nombre de filtros, orden y columnas sobre un `@List`.

```
@List
@SavedViews(scope = SavedViewScope.USER)
public class ListadoPedidos { ... }
```

- El usuario guarda «Pedidos urgentes sin confirmar» una vez y la recupera con un clic.
- `scope` distingue vistas privadas del usuario de vistas compartidas por equipo.
- Cada vista guardada debería ser **deep-linkable** (`@Route` que codifica el filtro), atacando Filter Amnesia por persistencia y por enlace.

Atajos de teclado

PARCIAL

Hoy `@Trigger(type=OnEnter)` cubre el envío con Enter. Para honrar de verdad *keyboard-first* faltaría una primitiva de atajos por acción (p. ej. un atributo `shortcut` en `@Action`) y, idealmente, una paleta de comandos. Se deja señalado como dirección, no como diseño cerrado.

7 Checklist de cierre de beta

Reconciliado con el catálogo. Estado a fecha de este documento.

ÍTEM	ESTADO	PRIMITIVA
Deep linking	EXISTE	@Route · @Routes · @HomeRoute · @BaseRoute
Split view	EXISTE	@SplitLayout + @MasterDetail
Entity pickers	EXISTE	@Lookup · @Composition
Wizard	EXISTE	@WizardCompletionAction + pasos validados
Jobs largos / async	EXISTE	@Action(background, sse) + @Trigger
Retry / recover	PARCIAL	@Action(confirmationRequired) + @Trigger(OnError)
Atajos de teclado	PARCIAL	@Trigger(OnEnter) — faltan atajos generales
Autosave	FALTA	@Autosave (propuesta)
Saved views	FALTA	@SavedViews (propuesta)
Timeline	FALTA	@Timeline (propuesta)
Task-centric UX	COMPOSICIÓN	Guía de diseño, sin primitiva nueva

Leyenda · **Existe**: disponible en v3 · **Parcial**: cubierto en parte, conviene reforzar · **Falta**: requiere primitiva nueva antes de la beta.